Joker of Academics

VRAM is Too Expensive So We Use Graduate Students to Train LLMs: The GUON Optimizer

Prof. Pig[1,*] , Student Piggy[2](zhuyihe163@163.com)

[1]School of Artificial Hype, Yeji University, Guankou City 100000, China
[2]School of Human Computation, Yeji University, Guankou City 100000, China

Joker of Academics | Cite This | Read Online | Metrics & More

Artical Type: Chaotic Review

Abstract

Scaling large language models (LLMs) confronts a severe memory wall crisis, with modern optimizers easily hoarding terabytes of historical states. Facing a dried-up funding account, we propose **Guon(Graduate Unpaid Optimization Nodes)**—a stateless second-order optimization paradigm driven entirely by carbon-based compute power. On the theory side, we biologically reconstruct the concept of space-time duality : since we cannot afford the VRAM to accumulate momentum across time , we densely pack cheap graduate students to fill the physical space, forcing them to manually perform FP32 gradient accumulation on scratch paper. On the systems side, we design a streaming offload and two-stream ping-pong schedule : the GPU is only responsible for spitting out single-layer gradients, which are immediately output as paper scrolls by an old dot-matrix printer for asynchronous offload. Group A students frantically calculate the Newton-Schulz iteration formula on blackboards to complete matrix orthogonalization , while Group B students sprint down the hallway to prefetch the printed sheets of the next layer. Miraculously, manual calculation errors caused by extreme sleep deprivation and sweat dripping on the paper perfectly serve as controlled perturbations to stabilize the spectral operator. Evaluations show that GUON successfully slashes the static memory requirement of a 70B model by roughly 8x (down to 140 GB). This study proves that as long as the supervisor's whip cracks fast enough, the computing potential of graduate students can completely push the training-time memory closer to the inference lower bound, with the only system bottlenecks being the reimbursement limit for A4 paper and the medical treatment costs for carpal tunnel syndrome.

Keywords: large language models; optimizer; second-order; memory; stateless training

1. Introduction

The parameter scale of LLMs continues to grow, making GPU memory capacity an increasingly dominant constraint for training.

Unlike inference—which primarily stores model weights—training must additionally store gradients and optimizer states; for modern adaptive methods, these parameter-sized persistent states often dominate memory. For instance, AdamW maintains first- and second-moment statistics [10, 12], while spec-

*Corresponding author: Prof. Pig.

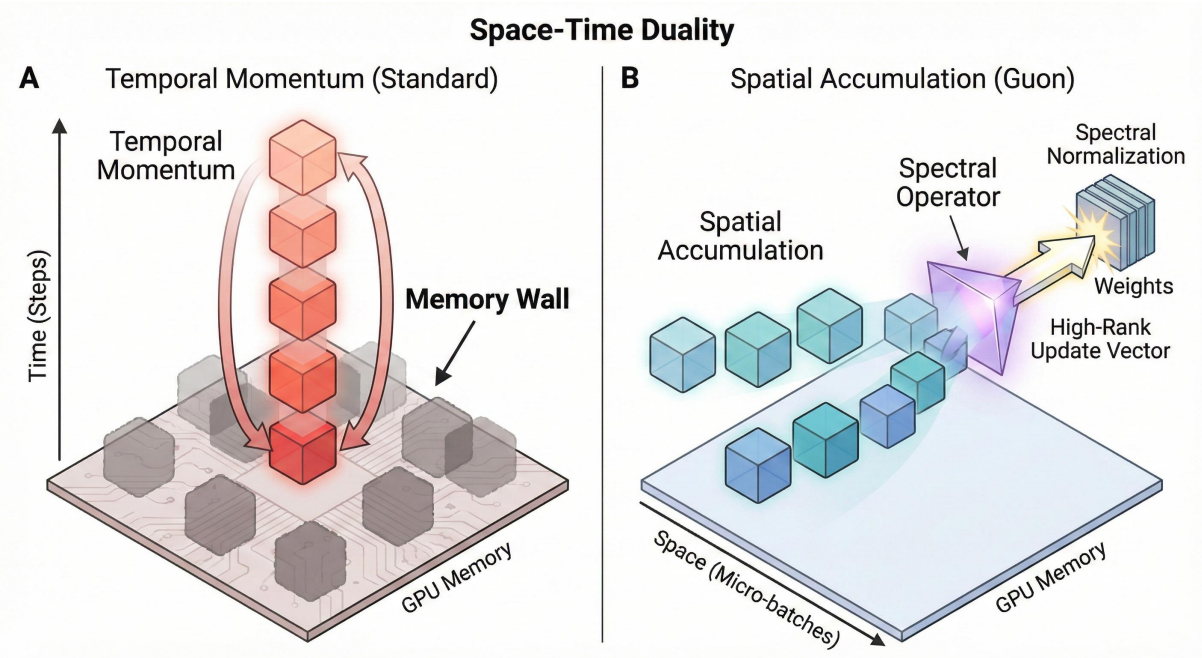

Figure 1: **A conceptual illustration of space–time duality. (A) Time-axis momentum (prior paradigm):** conventional optimizers (e.g., Muon) build rank and stability by recursively accumulating history along the time axis, inducing a *memory wall* since large historical states must reside on GPU. **(B) Space-axis accumulation (Guon):** Guon leverages the duality by accumulating gradients along the space axis (micro-batches). These instantaneous gradients pass through a **spectral operator** (purple prism) for just-in-time geometric shaping, enabling a stateless and streaming update path that reduces GPU memory usage to the inference scale.

tral/orthogonalization methods (e.g., Muon) typically rely on matrix momentum to increase the effective rank of update directions [1]. This *training–inference memory gap* raises the hardware threshold and limits the accessibility and reproducibility of method and hyperparameter exploration.

To mitigate memory pressure, prior systems work attacks the problem from different angles: e.g., offloading optimizer states to host memory and executing them in a pipelined fashion [14], or quantizing optimizer states to reduce constant factors [5]. However, these approaches typically do not change the fundamental dependence on *cross-step state tracking*: states are still kept persistently, merely compressed or relocated. We instead ask a more fundamental question: *can we remove parameter-sized persistent states at the algorithmic level while preserving effective optimization dynamics, thereby compressing training-time static memory toward the inference lower bound?*

Our central insight is **space–time duality**: for matrix-structured updates, the *rank-completion* role of momentum—accumulating low-rank gradients over time—can, under certain conditions, be replaced by gradient accumulation over micro-batches within a single update. By accumulating more micro-batches per update, one can increase the effective rank of the input to spectral operators and reduce noise in a statistical sense. This enables matrix-wise spectral normalization/orthogonalization to perform geometric shaping within a single step, without requiring long-lived history statistics to build element-wise preconditioners. This view is consistent with large-batch training experience: when the effective batch size is sufficiently large and paired with appropriate learning-rate scaling and normalization, training can remain stable [18, 19].

Building on this insight, we propose **Guon**, a stateless second-order optimization paradigm for large-model training. Guon restructures each update cycle into three pipelineable stages: (i) injecting controlled perturbations in the forward pass to stabilize stateless spectral operators; (ii) offloading gradients layer-by-layer during backpropagation to CPU memory and accumulating them in FP32 while freeing GPU gradient buffers early; and (iii) prefetching ac-

cumulated gradients back to a temporary GPU buffer layer-by-layer during the optimizer step, executing stabilized Newton–Schulz spectral updates, and immediately releasing the buffers. With a two-stream ping-pong schedule, transfer and compute overlap in steady state, compressing peak gradient residency to the layer level.

Given the high cost of end-to-end training at extreme scale, we combine **theoretical analysis** with **analytical evaluation** to characterize Guon's benefits and applicability. Section 3 analyzes the stability mechanism under low-rank inputs and derives conditions for statelessness; Section 4 presents an implementable streaming pipeline and pseudocode; Section 5 builds a first-principles performance model to quantify memory savings and masking conditions.

Our main contributions are:

1. **Theory:** we introduce the space–time duality perspective, clarifying how gradient accumulation can replace momentum's structural rank-completion role in the stateless setting, and analyze stabilization for Newton–Schulz spectral operators.

2. **Algorithm:** we propose a stateless matrix-wise spectral update framework combining controlled perturbations with stabilized Newton–Schulz iterations to achieve within-step geometric shaping.

3. **System:** we design a streaming offload and two-stream ping-pong prefetch pipeline that reduces GPU-resident states and peak gradient residency to the layer level.

4. **Evaluation:** we develop an analytical performance model to quantify Guon's potential gains and boundaries from memory and masking constraints.

2. Related Work

This section reviews relevant method families and sets up comparisons for later analysis. Broadly, existing approaches fall into two classes: (i) element-wise adaptive first-order methods (e.g., Adam/AdamW [10, 12], RMSProp [15], Adagrad [6]); (ii) matrix-wise preconditioning and spectral normalization methods (e.g., K-FAC [13], Shampoo [8], and orthogonalization/spectral methods [1]).

In what follows, we use AdamW and Muon as representatives of the two categories. This choice provides clear forms for exposing key mechanisms such as state dependence, spectral normalization, and rank completion; our conclusions extend to other element-wise adaptive methods and matrix-wise spectral/orthogonalization approaches.

2.1 Adaptive First-Order Optimizers (Adam and AdamW)

SGD serves as a stateless baseline with small memory overhead, but it often converges slowly under the ill-conditioned curvature typical of Transformers. Adam introduces per-parameter adaptive step sizes via first- and second-moment estimates [10]; AdamW decouples weight decay from gradient updates, better matching ℓ_2 regularization [12].

Let parameters be θ and gradients $g_t = \nabla_\theta \mathcal{L}(\theta_t)$. AdamW maintains

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t, \tag{2}$$

with bias corrections $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$. The update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t. \tag{3}$$

Limitations and challenges for statelessness.

Adam/AdamW's adaptivity fundamentally depends on element-wise variance statistics: v_t must accumulate across steps to become stable. Removing historical statistics and relying only on the current batch yields poor variance estimates and can degenerate toward sign/clipping-like updates, failing to address matrix-level anisotropy. Moreover, AdamW stores both m_t and v_t—two parameter-sized state tensors—leading to substantial GPU memory overhead at scale.

2.2 Spectral and Second-Order Optimizers (Muon and Related)

Second-order preconditioning methods aim to capture correlations across parameter dimensions. K-FAC [13] and Shampoo [8] build (approximate) curvature or second-moment preconditioners to accelerate convergence, but often incur high storage and matrix-inverse (or matrix-power) costs.

Spectral and orthogonalization methods instead normalize update directions at the matrix level. Recent progress includes scalable and stabilized Shampoo variants (e.g., SOAP) [9, 11], improved orthogonalized optimizers (e.g., ROOT, Muon++) [17, 21], low-bit matrix momentum (e.g., 8-bit Muon) [16], finer-grained normalization (e.g., NorMuon) [20], and spectral regularization / approximate second-

order baselines (e.g., PRISM, Full Gauss–Newton) [4, 7].

While these works form the broader context, our theoretical analysis focuses on Muon [1] to illuminate rank bottlenecks and stability mechanisms under stateless constraints.

Muon can be summarized as follows. For a layer with matrix gradient $G_t \in \mathbb{R}^{m\times n}$, Muon first accumulates a momentum buffer

$$U_t = \beta U_{t-1} + (1-\beta)G_t, \tag{4}$$

then applies a polar decomposition to obtain the orthogonal factor as a spectrally normalized update direction,

$$\tilde{G}_t = \mathrm{Orth}(U_t) \approx U_t\,(U_t^\top U_t)^{-\frac{1}{2}}. \tag{5}$$

Finally, parameters are updated as follows (reshape/transpose details omitted):

$$\theta_{t+1} = \theta_t - \eta\,\tilde{G}_t. \tag{6}$$

Newton–Schulz (NS) iteration.

In practice, $(U_t^\top U_t)^{-\frac{1}{2}}$ is often approximated on GPU via Newton–Schulz iterations. A common form normalizes

$$X_0 = \frac{1}{\|U_t\|_F}\,U_t^\top U_t, \tag{7}$$

and iterates

$$X_{k+1} = \frac{1}{2}X_k\,(3I - X_k^2), \qquad k = 0, 1, \ldots, K-1, \tag{8}$$

so that $X_K \approx (U_t^\top U_t)^{-\frac{1}{2}}$ and $\tilde{G}_t \approx U_t X_K$.

Structural state dependence and rank bottlenecks.

The numerical stability of spectral normalization is closely tied to the effective rank of the input. When U_t is nearly low rank, NS iterations tend to operate within a low-dimensional subspace and become more sensitive to null-space noise. Thus, retaining U_{t-1} is not merely for denoising; it plays a structural role in *rank completion*. Moreover, single-step (or single-microbatch) gradients often have effective rank limited by batch size; applying spectral normalization directly to low-rank inputs constrains optimization to a low-dimensional subspace.

2.3 Memory-Efficient Training Techniques

Approaches to reducing training memory usage can be broadly categorized as follows.

Low-bit optimizers.

Low-bit optimizers compress optimizer states (e.g., quantizing moments to INT8), substantially reducing constant factors [5]; however, the asymptotic order remains unchanged and cross-step states are still required.

Optimizer offloading.

Offloading moves optimizer states to CPU memory and uses asynchronous pipelines to reduce peak GPU memory [14], but may introduce CPU bottlenecks and heavy H2D/D2H transfers.

Difference from our method.

We distinguish *quantization/offloading* from *algorithmic statelessness*. Guon aims to remove persistent algorithmic states and replace them with streamable intermediate quantities (e.g., applying spectral normalization only to current accumulated gradients), pushing peak GPU memory close to the inference footprint.

2.4 Large-Batch Training and Gradient Accumulation

Large-batch training provides background for our space–time duality argument. Methods such as LARS [18] and LAMB [19] show that with extremely large effective batch sizes (e.g., 32k+), suitable learning-rate scaling and layer-wise adaptation can stabilize training and accelerate convergence.

Gradient accumulation is traditionally treated as an engineering compromise under memory constraints; here we view it as a mathematical operator. It aggregates gradients over micro-batches within a single update, increasing the chance that accumulated gradients in matrix layers exhibit higher effective rank, providing a more informative input to spectral normalization.

3. Theoretical Analysis

3.1 Formalizing Static Memory Overhead

We formalize the GPU-memory metric of interest under mixed-precision training and define the optimization boundary. Let the parameter vector be $\theta \in \mathbb{R}^N$. We decompose training-time GPU memory into four parts:

Optimizer	Momentum	Variance/Precond.	Resident gradients	Static overhead
SGD	0	0	$1 \times \|\theta\|$	1
SGD + Momentum	$1 \times \|\theta\|$	0	$1 \times \|\theta\|$	2
AdamW	$1 \times \|\theta\|$	$1 \times \|\theta\|$	$1 \times \|\theta\|$	3
Muon	$1 \times \|\theta\|$	0	$1 \times \|\theta\|$	2
Guon	0	0	≈ 0	≈ 0

Table 1: State complexity of representative optimizers (static overhead in units of $\|\theta\|$).

1. **Weights**: required for forward/backward and constitute a lower bound for both training and inference.

2. **Gradients**: produced by backpropagation and typically resident until an optimizer update is applied.

3. **Optimizer states**: cross-step statistics (e.g., AdamW's m_t, v_t or Muon's matrix momentum buffer).

4. **Activations**: dependent on batch size and sequence length.

We focus on **static memory** that scales with parameters (weights, gradients, and optimizer states). We assume activation checkpointing and related generic techniques are already employed [3] and do not treat activations as the primary target.

For cross-model comparison, we define static overhead in units of a single parameter tensor. Let $\|\theta\|$ denote the memory to store one copy of parameters (at the corresponding precision). Define

$$\text{Overhead} = \frac{\text{memory of resident gradients and optimizer states}}{\|\theta\|} \tag{9}$$

Guon's goal is to remove parameter-sized persistent optimizer states and to reduce gradient residency to the layer level via streaming.

3.2 State Complexity of Existing Optimizers

Table 1 summarizes resident tensors (in units of $\|\theta\|$) for typical optimizers.

Table 1 highlights that AdamW's main memory burden is two parameter-sized statistics (momentum and variance), while Muon's key state is the matrix momentum buffer that structurally contributes to rank completion. Guon's core idea is to replace momentum's time-axis rank completion with space-axis gradient accumulation, while avoiding resident gradients via streaming.

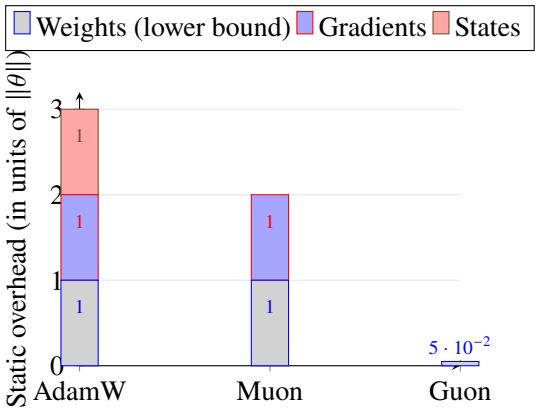

Figure 2: Illustration of static GPU-memory overhead for representative optimizers (consistent with Table 1). Guon only keeps layer-level temporary buffers, hence the model-wide overhead is approximately zero.

3.3 Rank Bottlenecks in Newton–Schulz

We explain Muon's structural dependence on momentum from spectral dynamics and numerical implementation. The core tension is that, although NS iterations are mathematically compatible with low-rank inputs, applying the spectral operator to single-microbatch low-rank gradients induces severe subspace blind spots and amplifies numerical noise under low precision.

Rank limitation of single-step gradients.

Consider a linear layer weight $W \in \mathbb{R}^{m \times n}$. A microbatch gradient can be written as

$$G = \sum_{i=1}^{B} a_i b_i^\top, \tag{10}$$

where B is the (micro)batch size. Thus

$$\operatorname{rank}(G) \le \min\{B, m, n\}. \tag{11}$$

When $B \ll \min\{m, n\}$, G is strongly rank-deficient.

Spectral dynamics of Newton–Schulz.

The orthogonalized update can be written as $\mathrm{Orth}(U) = U(U^\top U)^{-\frac{1}{2}}$. NS iteration effectively applies a polynomial map to singular values; after normalization, the core update

$$X_{k+1} = \tfrac{1}{2} X_k \left(3I - X_k^2\right) \tag{12}$$

can be understood as applying $f(\sigma) = \frac{1}{2}\sigma(3 - \sigma^2)$ to each singular value σ. Notably, $\sigma = 1$ is a stable fixed point, while $f(0) = 0$ implies that zero singular values remain zero in exact arithmetic.

Two issues with low-rank inputs.

Even if NS "runs," applying it directly to a low-rank single-step gradient causes:

1. **Subspace trap:** because $0 \mapsto 0$, components in $\mathrm{null}(G)$ remain zero; the update is confined to the low-dimensional subspace spanned by $\mathrm{range}(G)$.
2. **Noise amplification:** under BF16/FP16, tiny numerical noise may appear in theoretically zero singular directions; since $f(\sigma) > \sigma$ for $\sigma < 1$, such noise can be spuriously amplified.

Momentum as structural rank completion.

Muon uses matrix momentum

$$U_t = \beta U_{t-1} + (1 - \beta) G_t \tag{13}$$

to superpose different microbatch subspaces over time, increasing U_t's effective rank and improving spectral structure. Thus, the momentum buffer is structurally required for rank completion and stability.

3.4 Space–Time Duality: Replacing Momentum with Accumulation

The previous section shows that Muon's momentum buffer U_t plays a structural role: it increases effective rank via time-axis accumulation. Our key claim is that, without saving cross-step momentum, gradient accumulation within an update can deliver comparable rank completion.

Let microbatch gradients within update t be $G_{t,1}, \ldots, G_{t,K}$ where K is the number of accumulation steps. Define the accumulated gradient

$$\bar{G}_t = \frac{1}{K} \sum_{k=1}^{K} G_{t,k}. \tag{14}$$

Both time-axis momentum accumulation and space-axis gradient accumulation sum multiple (approximately low-rank) gradient matrices, increasing effective rank and improving conditioning of $U^\top U$. The difference is whether accumulation happens across updates (time) or within one update (space).

If the effective batch size $B_{\text{eff}} = KB_\mu$ is large enough such that the spectral-operator input is numerically "near full rank," then one can apply spectral normalization directly to $\bar{G}_t$ without cross-step momentum, achieving stability benefits comparable to time-axis rank completion. In practice, strict full-rank conditions are not required; rather, stability depends on effective rank and spectral gaps.

3.5 Physical Regularization: Stabilizing Stateless Spectral Normalization

Even with accumulation, $\bar{G}_t$ may have very small singular values, making $\bar{G}_t^\top \bar{G}_t$ ill-conditioned and destabilizing NS iterations. We therefore introduce a stabilization viewpoint decoupled from implementation details: before applying the spectral operator, inject controlled random perturbations (or an equivalent regularization) to lift the spectrum away from zero. Concretely, let

$$\tilde{G}_t = \bar{G}_t + \Delta_t, \tag{15}$$

where Δ_t is zero-mean noise with controlled variance. Injecting noise is often equivalent to adding Tikhonov-type regularization [2]; here it raises the spectral floor of the spectral-operator input and improves numerical stability when approximating $(\tilde{G}_t^\top \tilde{G}_t)^{-\frac{1}{2}}$.

3.6 Why Adam Cannot Be Stateless

This subsection clarifies why "stateless Adam" is fundamentally more difficult than stateless matrix-wise spectral updates.

Immediate consequence of removing states: preconditioner degeneration.

AdamW's key mechanism is the element-wise preconditioner

$$D_t = \mathrm{diag}\left((\sqrt{\hat{v}_t} + \epsilon)^{-1}\right). \tag{16}$$

Without cross-step statistics v_t, a natural substitute is to estimate variance from the current (accumulated) gradient, e.g., $\tilde{v}_t = \bar{g}_t \odot \bar{g}_t$. This makes the numerator and denominator tightly coupled, yielding highly unstable updates and, in the extreme, a sign-like behavior. In short, AdamW relies on long-horizon aver-

aging to obtain a usable variance estimate; removing history breaks the preconditioner.

Element-wise rescaling cannot resolve matrix anisotropy.

More importantly, even if a stable stateless estimate of $\tilde{v}_t$ existed, element-wise rescaling primarily reweights coordinates and does not explicitly reshape the spectral geometry of matrix gradients. For a linear layer with matrix gradient $G \in \mathbb{R}^{m\times n}$, steepest descent applies $-\eta G$. When curvature is highly anisotropic, updates can overshoot along directions associated with large singular values (large principal curvature), while progressing slowly along small-singular-value directions. Matrix-wise spectral operators (e.g., orthogonalization via $\mathrm{Orth}(\cdot)$) act on G as a whole and tend to "flatten" singular-value structure, directly targeting matrix anisotropy.

Takeaway.

Therefore, the stateless pathway advocated here is naturally aligned with matrix-wise spectral methods: the geometric transform can be performed within each update, whereas element-wise adaptive methods require persistent statistics to remain both stable and effective.

4. Methodology

We present Guon's end-to-end design that jointly addresses algorithmic statelessness and system-level streaming. GuonGuon consists of three stages: (1) perturbed forward for stability; (2) streaming offload and CPU accumulation; (3) just-in-time spectral update on GPU.

ubsectionGuon Cycle: Streaming, Stateless, Spectral

1. **Perturbed forward:** inject controlled perturbations during forward computation to realize physical regularization for stabilizing the spectral operator.

2. **Streaming offload and accumulation:** during backward, gradients are produced layer-by-layer; Guon asynchronously transfers each layer's gradient to pinned CPU memory, accumulates in FP32, and frees GPU gradient buffers early.

3. **Just-in-time spectral update:** during the optimizer step, Guon prefetches accumulated gradients back to a temporary GPU buffer per layer, runs stabilized NS iterations to approximate the orthogonal factor, updates parameters immediately, and releases buffers.

4.1 Algorithm Description

We describe Guon for a matrix-parameter layer $W \in \mathbb{R}^{m\times n}$; vectors or higher-order tensors can be reshaped accordingly.

4.2 System Design: Streaming and Overlap

We implement Guon as a "streaming optimizer": the GPU never needs to hold full-model gradients or optimizer states persistently; instead, it only keeps a small number of layer-level temporary buffers.

Gradient lifetime management.

In an autodiff framework, backpropagation produces per-layer gradients in topological order. Guon attaches hooks to each layer so that once a gradient is materialized, it is immediately scheduled for an asynchronous device-to-host (D2H) transfer into pinned CPU memory; the corresponding GPU gradient storage is then freed or reused as early as possible. The key is that offload and the remaining backward computation can overlap, achieving a "compute–transfer–free" streaming behavior.

CPU-side accumulators and precision.

The CPU maintains per-layer (or per-bucket) shadow buffers $A^{(\ell)}$ and performs accumulation in FP32 to mitigate rounding error and overflow across K micro-steps. During the optimizer step, $A^{(\ell)}$ is cast as needed for the GPU spectral operator.

Two-stream ping-pong overlap.

During the optimizer step, Guon overlaps host-to-device (H2D) prefetch with spectral computation using two CUDA streams: a transfer stream for asynchronous H2D and a compute stream for Newton–Schulz (NS) iterations and parameter updates. Two temporary GPU buffers (Buf A/B) are used in a ping-pong fashion so that while layer ℓ is being updated on the compute stream, layer $\ell+1$ is prefetched into the alternate buffer on the transfer stream.

Peak GPU-memory footprint.

With streaming offload and layer-wise JIT updates, peak GPU memory during training is dominated by

$$M_{\text{peak}} \approx M_{\text{weights}} + M_{\text{activations}} + M_{\text{buffer}}, \qquad (17)$$

Algorithm 1: Guon: Streaming Offload + Stateless Spectral Update (one update cycle)

Input: Layered parameters $\{W^{(\ell)}\}_{\ell=1}^{L}$; accumulation steps K; perturbation scale σ; NS iteration cap K_{NS}; learning rate η

Data: CPU-side FP32 accumulators $A^{(\ell)} \leftarrow 0$ (pinned memory)

```
// Stage I: Perturbed forward (no persistent weight modification)
for k ← 1 to K do
    Generate noise ξ^(ℓ) on-the-fly (seeded per layer) and use W̃^(ℓ) ← W^(ℓ) + σξ^(ℓ) in forward
    Run backward; obtain per-layer gradients G_k^(ℓ)
    // Stage II: Streaming offload and CPU accumulation (layer hooks)
    foreach layer ℓ (in reverse topological order) do
        Async D2H: copy G_k^(ℓ) to pinned CPU buffer
        Immediately free the corresponding GPU gradient buffer
        CPU accumulation: A^(ℓ) ← A^(ℓ) + G_k^(ℓ)

// Stage III: JIT spectral update (prefetch pipeline: compute ℓ while prefetching ℓ+1)
// Pipeline warm-up: prefetch layer 1 into Buf A
Transfer stream: async H2D prefetch A^(1) into GPU buffer B_A; record event E_1
foreach layer ℓ = 1, ..., L do
    // Ping-pong buffer selection
    if ℓ is odd then
        B_curr ← B_A
        B_next ← B_B
    else
        B_curr ← B_B
        B_next ← B_A
    // Wait for current layer's prefetch to complete (compute stream waits on event)
    Compute stream: wait for event E_ℓ
    // Key overlap: while computing layer ℓ, prefetch layer ℓ+1 on transfer stream
    if ℓ < L then
        Transfer stream: async H2D prefetch A^(ℓ+1) into B_next; record event E_ℓ+1
    // Spectral update for the current layer (while ℓ+1 is being transferred)
    Let Ḡ^(ℓ) ← B_curr/K and compute S ← (Ḡ^(ℓ))^⊤ Ḡ^(ℓ)
    Normalize X_0 ← S/α; run NS iterations (optionally early-stop) to obtain X ≈ S^(-1/2)
    Update: W^(ℓ) ← W^(ℓ) − η Ḡ^(ℓ) X; release B_curr
    Reset: A^(ℓ) ← 0
```

where M_{buffer} denotes the temporary workspace for a single layer's spectral update (including the GPU copy of $A^{(\ell)}$ and intermediate matrices in Newton–Schulz iterations). Since full-model gradients and optimizer states are no longer resident, M_{buffer} scales with the largest layer rather than the entire model, pushing the peak closer to the inference footprint.

4.3 Distributed Extension

We briefly discuss how Guon extends to data-parallel training. The key challenge is that gradients must be aggregated across devices, while Guon aims to keep GPU-resident communication buffers minimal and place parameter-sized state on the host side.

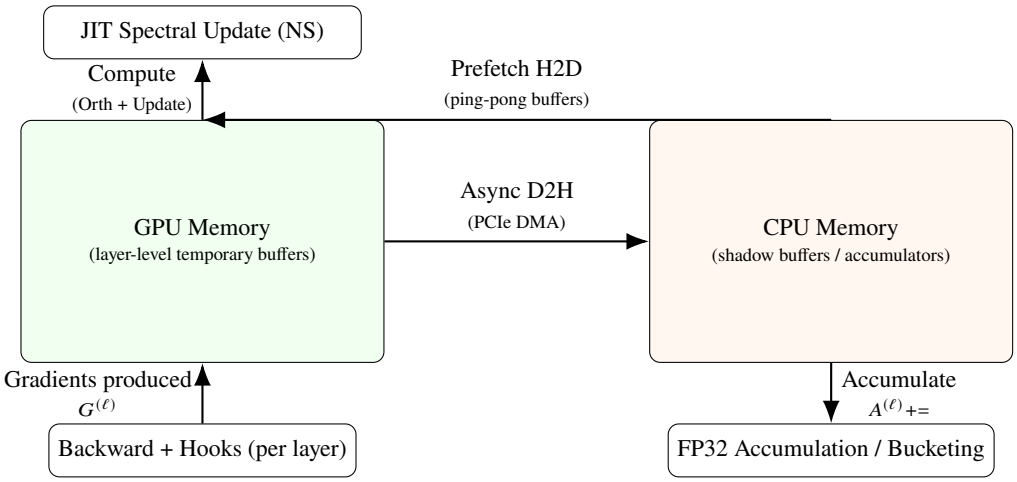

Figure 3: Guon's streaming offload and just-in-time (JIT) spectral update pipeline. Backward hooks offload gradients to CPU and free GPU buffers early; the optimizer step prefetches accumulated gradients back to GPU and performs spectral updates.

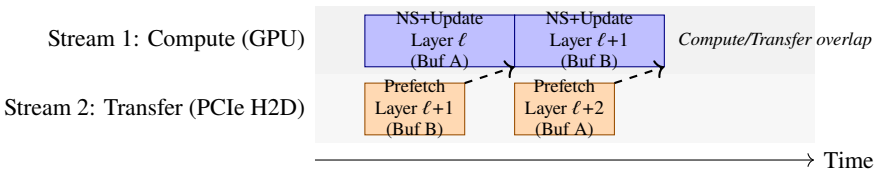

Figure 4: Two-stream ping-pong overlap (Nsight-style timeline): while the compute stream runs NS+Update for layer ℓ, the transfer stream concurrently prefetches layer ℓ+1 into the alternate buffer (Buf A/B), enabling "bubble-free" steady-state execution when compute dominates transfer.

Bucketing and CPU-side communication.

We group parameters/gradients into buckets and treat each bucket as the unit of accumulation and communication. After gradients are streamed to CPU accumulators, we can perform reductions and (where applicable) collective communication (e.g., all-reduce) on the CPU side per bucket. This shifts a substantial portion of communication staging away from GPU memory and decouples GPU spectral computation from communication.

Pipelining with spectral updates.

During the optimizer step, once a bucket's accumulated gradients have completed aggregation, it can be prefetched back to GPU and updated immediately. By overlapping bucket communication with spectral computation on neighboring buckets, the end-to-end step time becomes less sensitive to interconnect latency.

Discussion.

This extension does not modify Guon's core mathematical operator; it primarily changes system scheduling. Bucket granularity provides a knob to trade off communication overlap, CPU overhead, and GPU utilization.

5. Analytical Evaluation

Because end-to-end training at Llama-3-70B scale is extremely costly, we provide an analytical evaluation based on first-principles models. We quantify static memory savings, transfer/compute masking conditions, and communication implications.

5.1 Static Memory Savings

We focus on parameter-scaled static memory (weights, gradients, optimizer states) under mixed precision. Assume FP16 weights/gradients (2 bytes) and FP32 optimizer statistics (4 bytes).

AdamW.

A standard implementation typically stores: (i) FP16 parameters, (ii) FP16 gradients, (iii) FP32 master weights, (iv) FP32 first moment m, and (v) FP32 second moment v. Thus

$$M_{\text{AdamW}} \approx (2+2+4+4+4)N \text{ bytes} = 16N \text{ bytes}. \tag{18}$$

For $N \approx 70 \times 10^9$, this yields $\approx 1.12\,\text{TB}$.

Guon.

Guon removes parameter-sized persistent states and reduces gradient residency to the layer level. Ideally,

$$M_{\text{Guon}} \approx M_{\text{weights}} + M_{\text{buffer}} \approx 2N \text{ bytes} + M_{\text{buffer}}, \tag{19}$$

where M_{buffer} depends on the largest layer's temporary workspace rather than scaling linearly with N.

5.2 Masking Condition of the Two-Stream Pipeline

We model per-layer transfer and compute times. Let $A^{(\ell)} \in \mathbb{R}^{m\times n}$ be the accumulated gradient stored in FP32 on CPU and transferred back via PCIe. With effective H2D bandwidth B_{H2D} (bytes/s),

$$T_{\text{H2D}}^{(\ell)} \approx \frac{4mn}{B_{\text{H2D}}}. \tag{20}$$

Guon computes a stabilized NS approximation for $(\bar{G}^\top \bar{G})^{-1/2}$. Let K_{NS} be the iteration count and P_{eff} the effective device throughput. A coarse model is

$$T_{\text{NS}}^{(\ell)} \approx \frac{c\, K_{\text{NS}}\, \text{FLOPs}(m,n)}{P_{\text{eff}}}, \tag{21}$$

where c absorbs implementation details.

The pipeline is "bubble-free" in steady state if

$$T_{\text{NS}}^{(\ell)} \geq T_{\text{H2D}}^{(\ell+1)}, \tag{22}$$

so transfer for layer $\ell+1$ completes while computing layer ℓ.

5.3 Communication and Scalability (Discussion)

In data-parallel training, each update requires an all-reduce over parameter-sized gradients. If communication is organized over the accumulation window (e.g., bucket-wise aggregation on CPU followed by all-reduce), the average communication volume per micro-step scales as $\propto N/K$. Increasing K thus helps both statistical rank completion and system-level communication amortization, though too large K may affect optimization dynamics and requires appropriate learning-rate scaling.

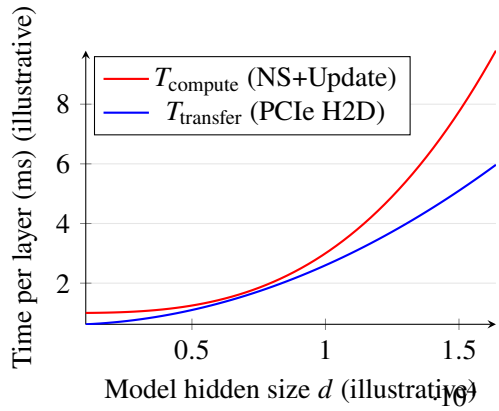

Figure 5: An illustrative masking analysis: as model width d increases, spectral-update compute (red) grows faster than PCIe transfer (blue), making it easier to satisfy $T_{\text{compute}} \geq T_{\text{transfer}}$ and achieve effective overlap. This figure conveys the qualitative trend; concrete values follow from hardware bandwidth and GEMM throughput.

6. Discussion and Limitations

Guon's primary contribution is to propose a stateless, matrix-wise spectral optimization paradigm paired with a streaming system design for breaking the memory wall. At the same time, it is important to clearly delineate the current validation scope and the conditions under which the method is most effective.

6.1 Convergence Validation: Theoretical Potential vs. Empirical Gap

Our analysis focuses on two questions: (i) whether statelessness is feasible for matrix-wise spectral operators, and (ii) why such operators can be advantageous under the anisotropic curvature typical of Transformers. In this framework, Guon uses space-axis accumulation to increase effective rank and reduce noise, replacing the time-axis rank-completion role played by momentum in Muon, thereby providing a numerically stable input to Newton–Schulz spectral normalization.

Nevertheless, theoretical arguments and a systems prototype do not fully substitute for large-scale, end-to-end convergence studies. For Llama-3-70B-scale models, complete token-level training curves, sensitivity to data domains and hyperparameters, and fair system comparisons against AdamW/Muon require a

mature implementation and substantial compute budget. We view this as a limitation of the present empirical scope rather than a direct refutation of feasibility.

More broadly, Guon's space-axis accumulation is tightly coupled to large-batch training: increasing the accumulation window K increases the effective batch size. Prior work (e.g., LARS and LAMB) indicates that with appropriate learning-rate scaling and normalization, large-batch training can remain stable and converge well. In this sense, Guon's design is consistent with established practice: matrix-wise spectral shaping acts directly on singular-value structure to mitigate anisotropy, while accumulation improves statistical stability and effective rank. Future work should systematically evaluate these interactions and characterize regimes in which the method matches or improves upon standard baselines.

6.2 Shifting System Bottlenecks: CPU and Bandwidth Dependence

Guon reduces GPU-resident, parameter-sized state by replacing persistence with streaming transfers and host-side accumulation. This shifts part of the system burden from GPU memory capacity to CPU resources and interconnect bandwidth.

CPU-side accumulation and bucketing overhead.

In Guon, the CPU is responsible for FP32 accumulation, bucket organization, and potentially CPU-side communication aggregation. If CPU memory bandwidth is insufficient, NUMA topology is unfavorable, or thread scheduling overhead is high, the host-side pipeline can become a throughput limiter and introduce GPU stalls. Thus, Guon is best matched to nodes with strong host-memory bandwidth and parallel CPU capability.

Interconnect bandwidth threshold.

Although Section 5 provides masking conditions and indicates that, in large-model regimes, spectral-update compute may dominate and hide PCIe transfers in steady state, this depends on interconnect generation and implementation details. On older or bandwidth-limited links, transfer time can become comparable to compute time, creating pipeline bubbles. Similarly, in distributed settings, insufficient overlap between communication and spectral updates can reduce the end-to-end gain.

In summary, Guon's advantage manifests as a trade-off among GPU memory, host-side resources, and interconnect bandwidth. Clarifying these conditions helps guide practical implementations and evaluation protocols.

7. Conclusion

This work revisits the memory wall in LLM training and attributes it to (i) the dependence of conventional optimizers on parameter-sized persistent states (e.g., AdamW moments or Muon's matrix momentum), and (ii) full-model gradient residency on GPU. We propose Guon, a stateless matrix-wise spectral optimization paradigm together with a streaming system design that reduces GPU-resident memory toward the inference lower bound.

7.1 From State Tracking to Geometric Shaping

Algorithmically, Guon's key idea is to shift the optimization viewpoint from cross-step state tracking to within-step geometric shaping. We formulate a space–time duality perspective: for matrix-structured updates, time-axis momentum accumulation can be replaced, under suitable conditions, by space-axis gradient accumulation within an update. This provides higher effective rank and improved numerical stability for Newton–Schulz-based spectral operators, enabling orthogonalized, spectrally shaped updates without persistent optimizer state.

7.2 System Design and Key Gains

System-wise, Guon restructures each update cycle into three pipelineable stages—perturbed forward, streaming offload with CPU-side accumulation, and just-in-time spectral updates on GPU. A two-stream ping-pong schedule overlaps H2D transfers with spectral computation and compresses peak gradient residency to the layer level. Our analytical evaluation indicates that for $N \approx 70 \times 10^9$ parameters, AdamW's static memory footprint is approximately $16N$ bytes (about 1.12 TB), whereas Guon's GPU-resident component approaches $2N$ bytes (about 140 GB) plus layer-level temporary buffers.

7.3 Outlook

Beyond a single method, Guon suggests a general training architecture: replacing persistent, parameter-sized state with streamable intermediates, and trading additional computation and scheduling complexity for improved memory accessibility. This direction may lower the barrier to exploring very large models on more modest hardware. Future work includes full end-to-end convergence studies at scale, improved overlap with distributed communication,

and co-design with learning-rate schedules and regularization hyperparameters.

References

[1] Jeremy Bernstein and Laker Newhouse. Modular duality in deep learning, 2024.

[2] Christopher M. Bishop. Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995.

[3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.

[4] Yuxin Chen, Yu Li, and Tianqi Chen. Prism: Principled spectral regularization for momentum sgd, 2025.

[5] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization, 2022.

[6] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011.

[7] Yibo Gao, Tianyi Zhang, Wenlong Xu, Wenxuan Zhou, Yu Li, and Tianqi Chen. Full gauss–newton for large language models, 2025.

[8] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[9] Chen-Yu Huang, Yu Tian, Yixin Chen, Li Xiang, Jianfei Chen, Yining Li, Miao Liu, and Tianqi Chen. Soap: Improving and stabilizing shampoo using adam, 2024.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2015.

[11] Tianjiao Liu, Qingxing Chu, Pengcheng Yu, Shuo Wang, Yuan Xu, Xinyang Zhang, Yuxin Chen, Yu Li, Minjia Zhang, and Eric P. Xing. Purifying shampoo: Easing shampoo's path to training large language models, 2025.

[12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[13] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature, 2015.

[14] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shaden Smith Yang, Minjia Zhang, Dong Li, Yuxiong Zhou, and Yuxiong He. ZeRO-Offload: Democratizing Billion-scale model training. In *USENIX Annual Technical Conference (ATC)*, 2021.

[15] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude, 2012. COURSERA: Neural Networks for Machine Learning.

[16] Yile Wang, Wenxuan Zhou, Wenlong Xu, Tianyi Zhang, Yu Li, and Tianqi Chen. 8-bit muon: Training llms with matrices of 8-bit momentum, 2025.

[17] Wenlong Xu, Wenxuan Zhou, Tianyi Zhang, Yile Wang, Yu Li, and Tianqi Chen. Root: A Robust and Efficient Optimizer for Orthogonalization and Training, 2025.

[18] Yang You, Igor Gitman, and Boris Ginsburg. Large batch training of convolutional networks, 2017.

[19] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes, 2019.

[20] Yuxin Zhang, Haotian Wang, Yu Li, and Tianqi Chen. Normuon: Neuron-wise normalization for orthogonalized optimization, 2025.

[21] Wenxuan Zhou, Wenlong Xu, Yile Wang, Tianyi Zhang, Yu Li, and Tianqi Chen. Muon++: A stable and efficient orthogonalized optimizer, 2025.